REST API · V1

Faktero API-Dokumentation

Kunden, Rechnungen, Angebote, Eingangsbelege, Bank, Webhooks und die Anbindung von KI-Assistenten — alles mit einem Schlüssel.

BASIS-URL	AUTHENTIFIZIERUNG
`https://faktero.sk/api/v1`	`Bearer fk_live_…`
FORMAT	**TESTMODUS**
`JSON (UTF-8)`	`fk_test_…`

ERSTE SCHRITTE

Einführung

Mit der Faktero REST API automatisieren Sie die Rechnungsstellung vollständig — Kunden anlegen, Rechnungen ausstellen, PDFs herunterladen, E-Mails senden und Webhooks zu Ereignissen empfangen. Alle Antworten sind im JSON-Format (UTF-8).

Authentifizierung

Authentifizieren Sie jede Anfrage mit dem Header `Authorization` und einem Bearer-API-Schlüssel. Schlüssel sind an ein bestimmtes Unternehmen und bestimmte Berechtigungen gebunden.

HTTP

```
Authorization: Bearer fk_live_xxxxxxxxxxxxxxxxxxxxxxxx
```

Geben Sie den Schlüssel niemals weiter und speichern Sie ihn nicht in öffentlichen Repositories. Einen verlorenen Schlüssel widerrufen Sie sofort im Bereich API-Schlüssel.

Test- und Live-Modus

Faktero unterscheidet zwei Schlüsseltypen anhand des Präfixes:

`fk_test_…` — Testmodus. Rechnungen werden nicht real versendet, E-Mails gehen in ein Testpostfach.

`fk_live_…` — Live-Modus. Aktionen sind real und die Rechnungsdaten werden in echte Dokumente geschrieben.

Fehler

Alle Fehlerantworten haben ein einheitliches Format:

```JSON
{
  "error": {
    "code": "validation_error",
    "message": "Human readable message",
    "details": {}
  }
}
```

Statuscodes

CODE	BEDEUTUNG
200	OK — Anfrage erfolgreich
201	Created — Ressource erstellt
400	Bad Request — fehlerhaftes Anfrageformat
401	Unauthorized — fehlender oder ungültiger API-Schlüssel
403	Forbidden — Schlüssel ohne Berechtigung
404	Not Found — Ressource existiert nicht
409	Conflict — Statuskonflikt (z. B. doppelte Nummer)
422	Unprocessable Entity — Validierungsfehler
500	Internal Server Error — Fehler auf Seiten von Faktero

Anfragelimits

Anfragelimits ergänzen wir vor dem öffentlichen Start der API. Für einen zuverlässigen Produktivbetrieb empfehlen wir, bei Antwort 429 die Anfrage mit zunehmendem Abstand zu wiederholen (exponentielles Backoff).

ENDPUNKTE

Kunden

POST	/api/v1/customers	— Kunden anlegen
GET	/api/v1/customers	— Kundenliste
GET	/api/v1/customers/{id}	— Kundendetail
PUT	/api/v1/customers/{id}	— Kunden bearbeiten

Rechnungen

POST	/api/v1/invoices	— Rechnung ausstellen
GET	/api/v1/invoices	— Rechnungsliste
GET	/api/v1/invoices/{id}	— Rechnungsdetail
PUT	/api/v1/invoices/{id}	— Bearbeiten (nur Entwürfe)
GET	/api/v1/invoices/{id}/pdf	— Temporärer signierter Link zum PDF
POST	/api/v1/invoices/{id}/send	— Per E-Mail senden
POST	/api/v1/invoices/{id}/mark-paid	— Als bezahlt markieren
POST	/api/v1/invoices/{id}/cancel	— Rechnung stornieren

Angebote

POST	/api/v1/quotes	— Angebot erstellen
GET	/api/v1/quotes	— Angebotsliste
GET	/api/v1/quotes/{id}	— Angebotsdetail
PUT	/api/v1/quotes/{id}	— Angebot bearbeiten
POST	/api/v1/quotes/{id}/convert	— In Rechnung umwandeln

Wiederkehrende Rechnungen

POST	/api/v1/recurring-invoices	— Vorlage erstellen
GET	/api/v1/recurring-invoices	— Vorlagenliste
GET	/api/v1/recurring-invoices/{id}	— Vorlagendetail
PUT	/api/v1/recurring-invoices/{id}	— Vorlage bearbeiten

Eingangsbelege und Bank

Für Buchhaltungsprogramme oder Kanzleien: Eingangsrechnungen und Kassenbons mit Kontierung (Vorkontierung, MwSt.-Kennzeichen, KV), Änderungen seit dem letzten Abruf und Markierung übergebener Belege — genau wie der Pohoda-Konnektor.

GET	`/api/v1/purchase-invoices`	— Eingangsrechnungen
GET	`/api/v1/purchase-invoices/{id}`	— Detail mit Positionen und Link zur Datei (1 h)
POST	`/api/v1/purchase-invoices/{id}/exported`	— Als übergeben markieren
GET	`/api/v1/receipts`	— Kassenbons
GET	`/api/v1/receipts/{id}`	— Kassenbon-Detail mit Link zur Datei
POST	`/api/v1/receipts/{id}/exported`	— Als übergeben markieren
POST	`/api/v1/unprocessed-documents`	— Beleg in Unverarbeitete Belege hochladen
GET	`/api/v1/unprocessed-documents`	— Unverarbeitete Belege und Erkennungsstatus
GET	`/api/v1/bank-accounts`	— Bankkonten des Unternehmens (IBAN, Bank, Saldo)
GET	`/api/v1/bank-transactions`	— Umsätze auf Bankkonten

Listenfilter

- `updated_since` — nur geändert seit (ISO 8601); die Liste ist sortiert nach `updated_at`, sodass Sie beim letzten Eintrag fortsetzen können.
- `issued_from`, `issued_to` — Rechnungsdatum (JJJJ-MM-TT).
- `exported=false` — noch nicht übergeben; `booked=true` — nur verbuchte (Eingangsrechnungen).
- `limit` (höchstens 200) und `offset`; die Antwort enthält `has_more` und `next_offset`.
- Bank: `date_from`, `date_to`, `bank_account_id`; `page` + `per_page` (oder `offset` + `limit`), höchstens 500 pro Seite. Die Antwort enthält `total`, `has_more` und `next_offset` — lesen Sie weiter, solange has_more true ist, sonst fehlen Ihnen Zeilen eines umsatzstarken Tages.

Bankkonten

`id` des Kontos ist genau die `bank_account_id`, die die Umsätze tragen. Auch getrennte Konten werden zurückgegeben (active: false), damit sich jede id aus älteren Umsätzen zuordnen lässt. Saldo und Zeitpunkt sind null, solange das Konto nicht mit der Bank synchronisiert wurde. Jeder Umsatz enthält zusätzlich iban und bank_name seines Kontos.

```JSON
GET /api/v1/bank-accounts
{
  "data": [
    {
      "id": "7c9a8242-5b05-4ab6-bc25-f543cf62079b",
      "name": "Tatra banka – predaj",
      "iban": "SK1211000000002612345678",
      "bank_name": "Tatra banka, a.s.",
      "currency": "EUR",
      "active": true,
      "balance": 1234.56,
      "balance_at": "2026-10-24T18:00:00Z"
    }
  ]
}
```

Ein unbekannter Pfad unter /api/v1/ liefert 404 mit dem Body `{"error": {"code":"not_found",…}}`.

Als übergeben markieren

```BASH
curl -X POST https://faktero.sk/api/v1/purchase-invoices/{id}/exported \
  -H "Authorization: Bearer fk_live_..." \
  -H "Content-Type: application/json" \
  -d '{ "exported": true, "accounting_number": "FP2026/0154" }'
```

`"exported": false` hebt die Markierung auf (der Beleg kann wieder bearbeitet werden).

Beleg hochladen

```BASH
curl -X POST https://faktero.sk/api/v1/unprocessed-documents \
  -H "Authorization: Bearer fk_live_..." \
  -H "Content-Type: application/json" \
  -d '{ "file_name": "faktura.pdf", "content_type": "application/pdf",
        "content_base64": "JVBERi0xLjQK...", "note": "zákazka Novák" }'
```

PDF, JPEG, PNG, WEBP, HEIC oder XML bis 14 MB. Der Beleg wird im Hintergrund erkannt (einige Dutzend Sekunden) und wartet unter Unverarbeitete Belege auf die Prüfung; den Status (`cita`, `vytazene`, `chyba`) liefert GET.

INTEGRATIONEN

Webhooks

Webhooks informieren Sie in Echtzeit über Ereignisse in Faktero. Den Endpoint legen Sie im Bereich Webhooks Ihres Kontos fest, wo Sie auch das Signing Secret erhalten.

Ereignisse

`invoice.created`	`invoice.sent`
`invoice.paid`	`invoice.cancelled`
`customer.created`	`bank_transaction.created`
`quote.created`	`quote.sent`
`quote.converted`	

Wann invoice.paid kommt

Bei jeder Änderung einer Rechnung auf bezahlt — nach dem Abgleich mit einem Bankumsatz (automatisch wie manuell), nach der Massenmarkierung in der Liste, per POST /api/v1/invoices/…/mark-paid und durch Statuswechsel im Rechnungsdetail. Bei mark-paid kommt es sofort, sonst innerhalb einer Minute. Stammt die Zahlung aus der Bank, enthält der Payload die bank_transaction_id des abgeglichenen Umsatzes.

bank_transaction.created

Kommt innerhalb einer Minute, nachdem der Umsatz in Faktero erscheint (Banksynchronisierung, Kontoauszugsimport, Bankbenachrichtigungen). Die Daten haben dieselbe Form wie ein Eintrag aus GET /api/v1/bank-transactions einschließlich iban und bank_name, Umsätze müssen also nicht abgefragt werden. Beim ersten Verbinden der Bank kommt für jeden geladenen Umsatz ein Ereignis.

JSON

```
{
  "event": "bank_transaction.created",
  "created_at": "2026-10-24T18:01:12Z",
  "data": {
    "id": "0b6f…",
    "bank_account_id": "7c9a8242-5b05-4ab6-bc25-f543cf62079b",
    "iban": "SK1211000000002612345678",
    "bank_name": "Tatra banka, a.s.",
    "booking_date": "2026-10-24",
    "amount": 1230.00,
    "currency": "EUR",
    "variable_symbol": "20260042",
    "counterparty": "Odberateľ s.r.o.",
    "matched_invoice_id": null
  }
}
```

Signaturprüfung

Jede Anfrage enthält den Header `X-Faktero-Signature` mit einer HMAC-SHA-256-Signatur des Anfragetexts, erstellt mit Ihrem *signing secret*.

NODE.JS

```
import crypto from "crypto";

function verifyFaktero(rawBody, signatureHeader, secret) {
  const expected = crypto
    .createHmac("sha256", secret)
    .update(rawBody, "utf8")
    .digest("hex");
  return crypto.timingSafeEqual(
    Buffer.from(signatureHeader),
    Buffer.from(expected),
  );
}
```

Wiederholung bei Fehlern

Antwortet Ihr Endpoint nicht innerhalb von 10 Sekunden mit 2xx, wiederholt Faktero den Versuch mit exponentiellem Abstand (1 min, 5 min, 30 min, 2 h, 12 h) — insgesamt max. 5 Versuche. Alle Versuche und ihren Status finden Sie unter Webhooks → Logs.

KI-Assistenten (MCP)

Faktero hat einen MCP-Server — über das Model Context Protocol verbinden sich Claude, ChatGPT, Cursor oder ein anderer KI-Assistent und arbeiten mit den Daten Ihres Unternehmens: Sie erfahren, wer Ihnen Geld schuldet, finden eine Rechnung, stellen eine neue aus oder sehen die Kontobewegungen. Claude.ai und ChatGPT melden sich über Faktero an, andere Clients mit einem API-Schlüssel.

- Adresse: `https://www.faktero.sk/api/mcp` (Streamable HTTP)
- Header: `Authorization: Bearer fk_live_…` — zum Ausprobieren verwenden Sie einen Testschlüssel.
- Werkzeuge: Rechnungen und ihr Detail, überfällige Rechnungen, Rechnungserstellung, Zahlungsvermerk, Kunden, Angebote, Eingangsrechnungen, Bankkonten und Umsätze.
- Der Assistent löscht nichts und versendet nichts — er stellt die Rechnung aus, der Versand per E-Mail oder als E-Rechnung liegt bei Ihnen. Schreibende Werkzeuge sind gekennzeichnet, damit der Client sie bestätigen lässt.

Claude.ai und ChatGPT

Fügen Sie einen eigenen Konnektor hinzu (in Claude: Einstellungen → Konnektoren, in ChatGPT: Einstellungen → Apps und Konnektoren) und geben Sie nur die Serveradresse ein. Ein Schlüssel ist nicht nötig — Faktero öffnet sich, Sie melden sich an, wählen das Unternehmen und bestätigen den Zugriff. Der Zugriff erscheint unter den API-Schlüsseln als „AI asistent (MCP)“ und lässt sich dort mit einem Klick widerrufen.

```text
https://www.faktero.sk/api/mcp
```

Claude Code

```bash
claude mcp add --transport http faktero https://www.faktero.sk/api/mcp \
  --header "Authorization: Bearer fk_live_..."
```

Andere Clients (JSON-Konfiguration)

```json
{
  "mcpServers": {
    "faktero": {
      "type": "http",
      "url": "https://www.faktero.sk/api/mcp",
      "headers": { "Authorization": "Bearer fk_live_..." }
    }
  }
}
```

BEISPIELE UND SUPPORT

Codebeispiele

Kunden anlegen

```curl
curl -X POST https://faktero.sk/api/v1/customers \
  -H "Authorization: Bearer fk_live_xxx" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Acme s.r.o.",
    "ico": "12345678",
    "ic_dph": "SK2020123456",
    "street": "Hlavná 1",
    "city": "Bratislava",
    "zip": "81101",
    "country": "SK",
    "email": "fakturacia@acme.sk"
  }'
```

JAVASCRIPT

```
const res = await fetch("https://faktero.sk/api/v1/customers", {
  method: "POST",
  headers: {
    "Authorization": "Bearer fk_live_xxx",
    "Content-Type": "application/json",
  },
  body: JSON.stringify({
    name: "Acme s.r.o.",
    ico: "12345678",
    email: "fakturacia@acme.sk",
  }),
});
const customer = await res.json();
```

PHP

```
<?php
$ch = curl_init("https://faktero.sk/api/v1/customers");
curl_setopt_array($ch, [
  CURLOPT_POST => true,
  CURLOPT_RETURNTRANSFER => true,
  CURLOPT_HTTPHEADER => [
    "Authorization: Bearer fk_live_xxx",
    "Content-Type: application/json",
  ],
  CURLOPT_POSTFIELDS => json_encode([
    "name" => "Acme s.r.o.",
    "ico" => "12345678",
    "email" => "fakturacia@acme.sk",
  ]),
]);
$customer = json_decode(curl_exec($ch), true);
```

Rechnung ausstellen

CURL

```
curl -X POST https://faktero.sk/api/v1/invoices \
  -H "Authorization: Bearer fk_live_xxx" \
  -H "Content-Type: application/json" \
  -d '{
    "customer_id": "cus_123",
    "issue_date": "2026-06-12",
    "due_date": "2026-06-26",
    "currency": "EUR",
    "items": [
      { "name": "Konzultácia", "quantity": 4, "unit": "hod",
        "unit_price": 75, "vat_rate": 23 }
    ]
  }'
```

PDF herunterladen

JAVASCRIPT

```
const res = await fetch(
  "https://faktero.sk/api/v1/invoices/inv_123/pdf",
  { headers: { Authorization: "Bearer fk_live_xxx" } }
);
const { signed_url } = await res.json();
// signed_url platí 5 minút – stiahnite alebo presmerujte používateľa
```

Rechnung per E-Mail senden

CURL

```
curl -X POST https://faktero.sk/api/v1/invoices/inv_123/send \
  -H "Authorization: Bearer fk_live_xxx" \
  -H "Content-Type: application/json" \
  -d '{ "recipient_email": "klient@example.com" }'
```

Beim ersten Versand muss das Rechnungsdatum von heute sein. Eine Rechnung mit älterem Datum lehnt die API mit 409 und dem Code issue_date_in_past ab (details enthält issue_date, today, new_due_date und can_update). Senden Sie die Anfrage erneut mit "update_issue_date": true — das Rechnungsdatum wird auf heute geändert, die Fälligkeit verschiebt sich um dieselbe Anzahl Tage und das Lieferdatum bleibt. Lässt sich das Datum nicht ändern (can_update: false — abgeschlossener Zeitraum oder Export in die Buchhaltung), wird die Rechnung erst mit "keep_issue_date": true mit dem ursprünglichen Datum versendet. Mahnungen und erneuter Versand werden nicht geprüft.

JSON

```
HTTP/1.1 409 Conflict
{
  "error": {
    "code": "issue_date_in_past",
    "message": "Dátum vystavenia nie je dnešný. …",
    "details": {
      "issue_date": "2026-10-20",
      "today": "2026-10-21",
      "new_due_date": "2026-11-04",
      "can_update": true
    }
  }
}

curl -X POST https://faktero.sk/api/v1/invoices/inv_123/send \
  -H "Authorization: Bearer fk_live_xxx" \
  -H "Content-Type: application/json" \
  -d '{ "recipient_email": "klient@example.com", "update_issue_date": true }'
```

Rechnung als bezahlt markieren

CURL

```
curl -X POST https://faktero.sk/api/v1/invoices/inv_123/mark-paid \
  -H "Authorization: Bearer fk_live_xxx"
```

Webhook empfangen (PHP)

PHP

```
<?php
$secret = getenv("FAKTERO_WEBHOOK_SECRET");
$payload = file_get_contents("php://input");
$sig = $_SERVER["HTTP_X_FAKTERO_SIGNATURE"] ?? "";
$expected = hash_hmac("sha256", $payload, $secret);

if (!hash_equals($expected, $sig)) {
  http_response_code(401);
  exit;
}

$event = json_decode($payload, true);
// $event["type"] === "invoice.paid" …
http_response_code(200);
```

Für Entwickler

Kostenloses Konto erstellen

API-Playground

Support kontaktieren